

Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральный государственный образовательный университет высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра ТС и ВС

Расчётно-графическая работа

По дисциплине: Базы данных

**Тема: Проектирование и реализация БД
«Мониторинг сети»**

Выполнил: Клишин Владимир
Евгеньевич

Группа: ИКС-431

Проверил: Брагин Кирилл Игоревич

1. Описание предметной области

Система мониторинга сети предназначена для учёта сетевой инфраструктуры, сбора метрик, регистрации аварий и управления оповещениями дежурным инженером.

Ключевые сущности:

- Узлы (nodes): серверы, роутеры, коммутаторы, базовые станции
- Метрики (metrics): показатели с узлов (CPU, ping, температура)
- Аварии (incidents): критические события с уровнем severity
- Оповещения (alerts): уведомления инженерам через email/SMS/Telegram
- Инженеры (engineers): дежурный персонал

Бизнес-процессы:

1. Сбор метрик с узлов
2. Обнаружение аварий при превышении порогов
3. Отправка оповещений дежурным
4. Реакция и закрытие инцидентов
5. Аналитика uptime и частоты аварий

2. Скрипт инициализации БД (schema.sql)

-- Схема БД "Мониторинг сети"

DROP TABLE IF EXISTS alerts, incidents, metrics, engineers, metric_types, nodes;

-- 1. типы метрик

```
CREATE TABLE metric_types (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(50) UNIQUE NOT NULL,  
  unit VARCHAR(20),  
  threshold_critical FLOAT,  
  threshold_warning FLOAT  
);
```

-- 2. Узлы сети

```
CREATE TABLE nodes (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  ip_address INET UNIQUE NOT NULL,  
  node_type VARCHAR(50) NOT NULL,  
  location VARCHAR(100),  
  status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'inactive', 'maintenance')),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 3. Собранные метрики

```
CREATE TABLE metrics (  
  id SERIAL PRIMARY KEY,  
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,  
  metric_type_id INTEGER NOT NULL REFERENCES metric_types(id),  
  value FLOAT NOT NULL CHECK (value >= 0),  
  recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 4. Дежурные инженеры

```
CREATE TABLE engineers (  
  id SERIAL PRIMARY KEY,  
  full_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  phone VARCHAR(20),  
  role VARCHAR(50) DEFAULT 'junior' CHECK (role IN ('junior', 'senior', 'team_lead')),  
  is_active BOOLEAN DEFAULT TRUE  
);
```

-- 5. Аварии

```
CREATE TABLE incidents (  
  id SERIAL PRIMARY KEY,  
  node_id INTEGER NOT NULL REFERENCES nodes(id),  
  metric_id INTEGER REFERENCES metrics(id),  
  severity VARCHAR(20) NOT NULL CHECK (severity IN ('critical', 'warning', 'info')),  
  status VARCHAR(20) DEFAULT 'open' CHECK (status IN ('open', 'acknowledged', 'resolved',  
'closed')),  
  description TEXT,  
  started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  resolved_at TIMESTAMP,  
  resolved_by INTEGER REFERENCES engineers(id),  
  CONSTRAINT chk_resolution_time CHECK (resolved_at IS NULL OR resolved_at > started_at)  
);
```

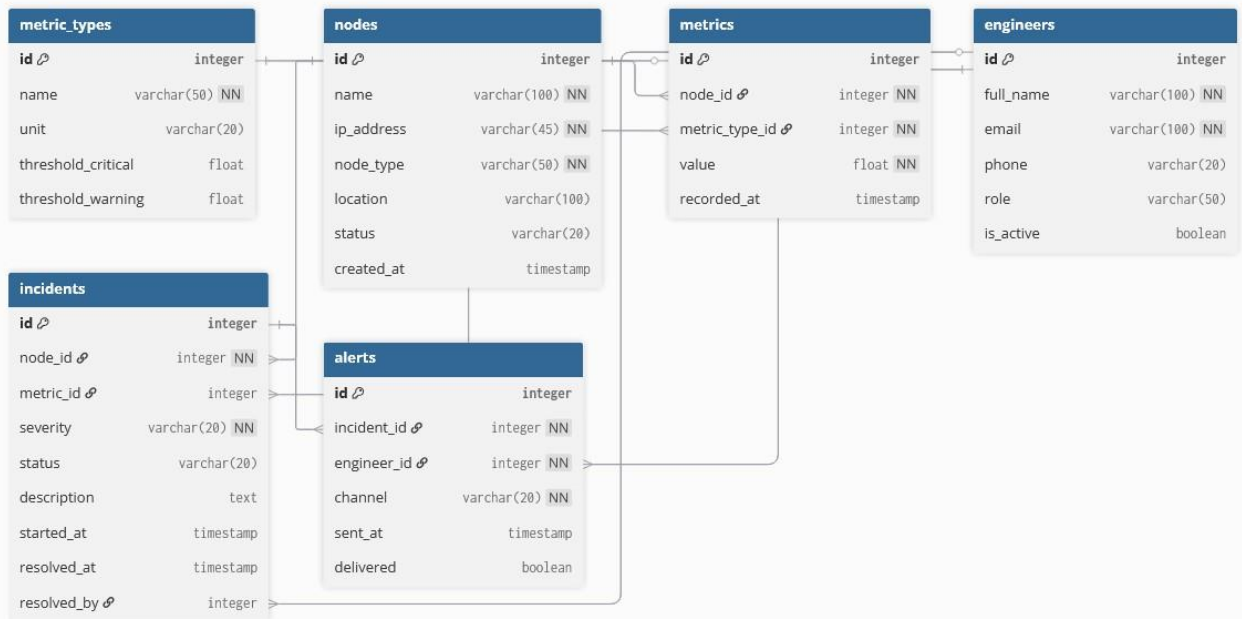
-- 6. Оповещения

```
CREATE TABLE alerts (  
  id SERIAL PRIMARY KEY,  
  incident_id INTEGER NOT NULL REFERENCES incidents(id) ON DELETE CASCADE,  
  engineer_id INTEGER NOT NULL REFERENCES engineers(id),  
  channel VARCHAR(20) NOT NULL CHECK (channel IN ('email', 'sms', 'telegram', 'MAX')),  
  sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  delivered BOOLEAN DEFAULT FALSE  
);
```

-- Индексы

```
CREATE INDEX idx_metrics_node_time ON metrics(node_id, recorded_at);  
CREATE INDEX idx_incidents_status_severity ON incidents(status, severity);  
CREATE INDEX idx_alerts_sent_at ON alerts(sent_at);
```

3. Графическая модель данных (ER-диаграмма)



dbdiagram.io

Основные связи между таблицами

nodes → metrics (1:N)

Один узел генерирует множество метрик. При удалении узла каскадно удаляются все его метрики (ON DELETE CASCADE).

nodes → incidents (1:N)

Один узел может иметь множество аварий. Узел нельзя удалить, если по нему есть зарегистрированные аварии (ON DELETE RESTRICT).

metric_types → metrics (1:N)

Один тип метрики используется во множестве записей. Тип нельзя удалить, если есть связанные метрики (ON DELETE RESTRICT).

metrics → incidents (0..1:1)

Одна метрика может породить одну аварию. Связь необязательная — авария может быть создана без привязки к конкретной метрике (metric_id допускает NULL).

incidents → alerts (1:N)

Одна авария порождает множество оповещений. При удалении аварии каскадно удаляются все её оповещения (ON DELETE CASCADE).

engineers → incidents (1:N)

Один инженер может закрыть множество аварий. При удалении инженера поле resolved_by сбрасывается в NULL (ON DELETE SET NULL).

engineers → alerts (1:N)

Один инженер получает множество оповещений. Инженера нельзя удалить, если на него есть активные оповещения (ON DELETE RESTRICT).

Наполнение базы данных (data.sql)

-- 1. Типы метрик (id: 1-4)

```
INSERT INTO metric_types (name, unit, threshold_critical, threshold_warning) VALUES
('cpu_usage', '%', 90.0, 75.0),
('memory_usage', '%', 95.0, 80.0),
('ping', 'ms', 500.0, 200.0),
('temperature', '°C', 80.0, 60.0);
```

-- 2. Узлы (id: 1-6)

```
INSERT INTO nodes (name, ip_address, node_type, location, status) VALUES
('Router-Moscow-01', '192.168.1.1', 'router', 'Москва, ЦОД-3', 'active'),
('Server-SP-02', '192.168.1.10', 'server', 'Санкт-Петербург', 'active'),
('Switch-Kazan-01', '192.168.2.1', 'switch', 'Казань', 'active'),
('BaseStation-Nov-01', '10.0.1.1', 'base_station', 'Новосибирск', 'inactive'),
('Router-Moscow-02', '192.168.1.2', 'router', 'Москва, ЦОД-3', 'maintenance'),
('Server-Ekat-01', '192.168.3.10', 'server', 'Екатеринбург', 'active');
```

-- 3. Инженеры (id: 1-3)

```
INSERT INTO engineers (full_name, email, phone, role, is_active) VALUES
('Иванов Петр', 'ivanov@company.ru', '+79001234567', 'senior', true),
('Сидорова Анна', 'sidorova@company.ru', '+79007654321', 'junior', true),
('Петров Дмитрий', 'petrov@company.ru', '+79005551122', 'team_lead', true);
```

-- 4. Метрики (id: 1-18)

```
INSERT INTO metrics (node_id, metric_type_id, value, recorded_at) VALUES
-- Router-Moscow-01 (node_id=1): id 1-3
(1, 1, 45.5, '2026-05-15 10:00:00'),
(1, 3, 12.0, '2026-05-15 10:00:00'),
(1, 4, 55.0, '2026-05-15 10:00:00'),
-- Server-SP-02 (node_id=2): id 4-6
(2, 1, 92.3, '2026-05-15 10:05:00'),
(2, 2, 88.0, '2026-05-15 10:05:00'),
(2, 3, 45.0, '2026-05-15 10:05:00'),
-- Switch-Kazan-01 (node_id=3): id 7-9
(3, 1, 30.0, '2026-05-15 10:10:00'),
(3, 3, 8.0, '2026-05-15 10:10:00'),
(3, 4, 42.0, '2026-05-15 10:10:00'),
-- BaseStation-Nov-01 (node_id=4): id 10-12
(4, 1, 10.0, '2026-05-10 08:00:00'),
(4, 3, 600.0, '2026-05-10 08:00:00'),
(4, 4, 85.0, '2026-05-10 08:00:00'),
-- Server-Ekat-01 (node_id=6): id 13-15
(6, 1, 67.0, '2026-05-15 10:15:00'),
(6, 2, 45.0, '2026-05-15 10:15:00'),
(6, 3, 25.0, '2026-05-15 10:15:00'),
-- Дополнительные для аналитики: id 16-18
(1, 1, 78.0, '2026-05-14 10:00:00'),
(1, 1, 95.0, '2026-05-13 10:00:00'),
(2, 1, 88.0, '2026-05-14 10:05:00');
```

-- 5. Аварии

```
INSERT INTO incidents (node_id, metric_id, severity, status, description, started_at, resolved_at, resolved_by) VALUES
(2, 5, 'critical', 'open', 'CPU превысил 90%', '2026-05-15 10:05:00', NULL, NULL),
(4, 12, 'critical', 'acknowledged', 'Базовая станция недоступна, высокая температура', '2026-05-10 08:00:00', NULL, NULL),
(1, 16, 'warning', 'closed', 'CPU высокий, но в пределах', '2026-05-14 10:00:00', '2026-05-14 12:00:00', 1),
(1, 17, 'critical', 'closed', 'CPU критический, перезагрузка', '2026-05-13 10:00:00', '2026-05-13 10:30:00', 1),
(2, 18, 'warning', 'closed', 'CPU загружен', '2026-05-14 10:05:00', '2026-05-14 11:00:00', 2),
(3, 8, 'warning', 'closed', 'Ping нестабильный', '2026-05-15 10:10:00', '2026-05-15 10:20:00', 1),
(6, 14, 'warning', 'open', 'Memory usage растёт', '2026-05-01 09:00:00', NULL, NULL),
(3, 9, 'info', 'closed', 'Температура в норме после обслуживания', '2026-05-15 10:10:00', '2026-05-15 10:15:00', 3);
```

-- 6. Оповещения (incident_id только 1-8!)

```
INSERT INTO alerts (incident_id, engineer_id, channel, sent_at, delivered) VALUES
(1, 1, 'email', '2026-05-15 10:06:00', true),
(1, 1, 'sms', '2026-05-15 10:06:30', true),
(2, 3, 'telegram', '2026-05-10 08:05:00', true),
(2, 1, 'email', '2026-05-10 08:05:00', false),
(3, 1, 'email', '2026-05-14 10:01:00', true),
(4, 1, 'sms', '2026-05-13 10:01:00', true),
(5, 2, 'email', '2026-05-14 10:06:00', true),
(6, 1, 'telegram', '2026-05-15 10:11:00', true),
(7, 2, 'email', '2026-05-01 09:05:00', false),
(8, 3, 'email', '2026-05-15 10:12:00', true);
```

Ограничения целостности и стратегия индексации

1. PRIMARY KEY

Каждая таблица содержит первичный ключ `id` типа `SERIAL` — уникальный идентификатор записи, автоматически генерируемый PostgreSQL.

2. FOREIGN KEY

Внешние ключи обеспечивают связи между таблицами:

- `metrics.node_id` → `nodes.id` (ON DELETE CASCADE)
- `metrics.metric_type_id` → `metric_types.id`
- `incidents.node_id` → `nodes.id`
- `incidents.metric_id` → `metrics.id`
- `incidents.resolved_by` → `engineers.id` (ON DELETE SET NULL)
- `alerts.incident_id` → `incidents.id` (ON DELETE CASCADE)
- `alerts.engineer_id` → `engineers.id`

3. UNIQUE

- `nodes.ip_address` — IP-адрес узла уникален в сети
- `metric_types.name` — название типа метрики не дублируется
- `engineers.email` — почта инженера уникальна

4. NOT NULL

Обязательные поля: `name`, `ip_address`, `node_type`, `value`, `severity`, `full_name`, `email` и другие — предотвращают появление неполных записей.

5. CHECK

- `nodes.status` IN ('active', 'inactive', 'maintenance') — допустимые статусы узла
- `incidents.severity` IN ('critical', 'warning', 'info') — уровни критичности
- `incidents.status` IN ('open', 'acknowledged', 'resolved', 'closed') — статусы аварии
- `alerts.channel` IN ('email', 'sms', 'telegram') — каналы оповещений
- `engineers.role` IN ('junior', 'senior', 'team_lead') — уровни инженеров
- `metrics.value` >= 0 — значение метрики неотрицательно
- `resolved_at` IS NULL OR `resolved_at` > `started_at` — авария не может закрыться раньше нач

SQL-запросы(queries.sql)

-- 1.Список всех узлов с типом и статусом
-- Задача: получить полный реестр оборудования

```
SELECT
    id,
    name,
    ip_address,
    node_type,
    location,
    status
FROM nodes
ORDER BY node_type, name;
```

-- 2.Узлы в статусе 'inactive'
-- Задача: найти недоступное оборудование

```
SELECT
    id,
    name,
    ip_address,
    node_type,
    location
FROM nodes
WHERE status = 'inactive';
```

-- 3. Средняя загрузка CPU по узлам
-- Задача: узнать среднюю нагрузку процессора

```
SELECT
    n.name AS node_name,
    AVG(m.value) AS avg_cpu
FROM nodes n
JOIN metrics m ON n.id = m.node_id
JOIN metric_types mt ON m.metric_type_id = mt.id
WHERE mt.name = 'cpu_usage'
GROUP BY n.id, n.name
ORDER BY avg_cpu DESC;
```


-- 4. Узлы с количеством аварий > 1
-- Задача: найти проблемные узлы (часто ломаются)

```
SELECT
    n.name,
    COUNT(i.id) AS incident_count
FROM nodes n
JOIN incidents i ON n.id = i.node_id
GROUP BY n.id, n.name
HAVING COUNT(i.id) > 1;
```

-- 5. Аварии с информацией об узле и метрике
-- Задача: полная картина по каждой аварии

```
SELECT
    i.id,
    n.name AS node_name,
    n.ip_address,
    mt.name AS metric_type,
    m.value AS metric_value,
    i.severity,
    i.status,
    i.description,
    i.started_at
FROM incidents i
JOIN nodes n ON i.node_id = n.id
LEFT JOIN metrics m ON i.metric_id = m.id
LEFT JOIN metric_types mt ON m.metric_type_id = mt.id
ORDER BY i.started_at DESC;
```

-- 6.Аварии с информацией об узле
-- Задача: полная информация об авариях с названиями узлов

```
WITH incident_nodes AS (  
    SELECT  
        i.id,  
        i.severity,  
        i.status,  
        i.description,  
        i.started_at,  
        n.name AS node_name  
    FROM incidents i  
    JOIN nodes n ON i.node_id = n.id  
)  
SELECT  
    id AS incident_id,  
    node_name,  
    severity,  
    status,  
    description,  
    started_at  
FROM incident_nodes  
ORDER BY started_at DESC;
```

-- 7. Топ-5 узлов по среднему пингу
-- Задача: найти узлы с наихудшей сетевой задержкой

```
SELECT  
    n.name AS node_name,  
    AVG(m.value) AS avg_ping_ms  
FROM nodes n  
JOIN metrics m ON n.id = m.node_id  
JOIN metric_types mt ON m.metric_type_id = mt.id  
WHERE mt.name = 'ping'  
GROUP BY n.id, n.name  
ORDER BY avg_ping_ms DESC  
LIMIT 5;
```

-- 8.Аварии за 2026 год
-- Задача: статистика динамики аварийности

```
SELECT
    started_at,
    description,
    severity,
    status
FROM incidents
WHERE started_at >= '2026-01-01' AND started_at < '2027-01-01'
ORDER BY started_at;
```

-- 9. Сколько оповещений у каждого инженера
-- Задача: нагрузка на инженеров

```
SELECT
    e.full_name,
    COUNT(a.id) AS alert_count
FROM engineers e
LEFT JOIN alerts a ON e.id = a.engineer_id
GROUP BY e.id, e.full_name
ORDER BY alert_count DESC;
```

-- 10. Открытые критические аварии
-- Задача: требуют немедленного вмешательства

```
SELECT
    i.id,
    n.name AS node_name,
    i.description,
    i.started_at
FROM incidents i
JOIN nodes n ON i.node_id = n.id
WHERE i.status = 'open'
AND i.severity = 'critical'
ORDER BY i.started_at;
```

id	name	ip_address	node_type	location	status
4	BaseStation-Nov-01	10.0.1.1	base_station	Новосибирск	inactive
1	Router-Moscow-01	192.168.1.1	router	Москва, ЦОД-3	active
5	Router-Moscow-02	192.168.1.2	router	Москва, ЦОД-3	maintenance
6	Server-Ekat-01	192.168.3.10	server	Екатеринбург	active
2	Server-SP-02	192.168.1.10	server	Санкт-Петербург	active
3	Switch-Kazan-01	192.168.2.1	switch	Казань	active

(6 rows)

id	name	ip_address	node_type	location
4	BaseStation-Nov-01	10.0.1.1	base_station	Новосибирск

(1 row)

node_name	avg_cpu
Server-SP-02	90.15
Router-Moscow-01	72.83333333333333
Server-Ekat-01	67
Switch-Kazan-01	30
BaseStation-Nov-01	10

(5 rows)

name	incident_count
Server-SP-02	2
Switch-Kazan-01	2
Router-Moscow-01	2

(3 rows)

id	node_name	ip_address	metric_type	metric_value	severity	status	description	started_at
6	Switch-Kazan-01	192.168.2.1	ping	8	warning	closed	Ping нестабильный	2026-05-15 10:10:00
8	Switch-Kazan-01	192.168.2.1	temperature	42	info	closed	Температура в норме после обслуживания	2026-05-15 10:10:00
1	Server-SP-02	192.168.1.10	memory_usage	88	critical	open	CPU превысил 90%	2026-05-15 10:05:00
5	Server-SP-02	192.168.1.10	cpu_usage	88	warning	closed	CPU загружен	2026-05-14 10:05:00
3	Router-Moscow-01	192.168.1.1	cpu_usage	78	warning	closed	CPU высокий, но в пределах	2026-05-14 10:00:00
4	Router-Moscow-01	192.168.1.1	cpu_usage	95	critical	closed	CPU критический, перезагрузка	2026-05-13 10:00:00
2	BaseStation-Nov-01	10.0.1.1	temperature	85	critical	acknowledged	Базовая станция недоступна, высокая температура	2026-05-10 08:00:00
7	Server-Ekat-01	192.168.3.10	memory_usage	45	warning	open	Memory usage растёт	2026-05-01 09:00:00

(8 rows)

incident_id	node_name	severity	status	description	started_at
6	Switch-Kazan-01	warning	closed	Ping нестабильный	2026-05-15 10:10:00
8	Switch-Kazan-01	info	closed	Температура в норме после обслуживания	2026-05-15 10:10:00
1	Server-SP-02	critical	open	CPU превысил 90%	2026-05-15 10:05:00
5	Server-SP-02	warning	closed	CPU загружен	2026-05-14 10:05:00
3	Router-Moscow-01	warning	closed	CPU высокий, но в пределах	2026-05-14 10:00:00
4	Router-Moscow-01	critical	closed	CPU критический, перезагрузка	2026-05-13 10:00:00
2	BaseStation-Nov-01	critical	acknowledged	Базовая станция недоступна, высокая температура	2026-05-10 08:00:00
7	Server-Ekat-01	warning	open	Memory usage растёт	2026-05-01 09:00:00

(8 rows)

node_name	avg_ping_ms
BaseStation-Nov-01	600
Server-SP-02	45
Server-Ekat-01	25
Router-Moscow-01	12
Switch-Kazan-01	8

(5 rows)

started_at	description	severity	status
2026-05-01 09:00:00	Memory usage растёт	warning	open
2026-05-10 08:00:00	Базовая станция недоступна, высокая температура	critical	acknowledged
2026-05-13 10:00:00	CPU критический, перезагрузка	critical	closed
2026-05-14 10:00:00	CPU высокий, но в пределах	warning	closed
2026-05-14 10:05:00	CPU загружен	warning	closed
2026-05-15 10:05:00	CPU превысил 90%	critical	open
2026-05-15 10:10:00	Ping нестабильный	warning	closed
2026-05-15 10:10:00	Температура в норме после обслуживания	info	closed

(8 rows)

full_name	alert_count
Иванов Петр	6
Петров Дмитрий	2
Сидорова Анна	2

(3 rows)

id	node_name	description	started_at
1	Server-SP-02	CPU превысил 90%	2026-05-15 10:05:00

(1 row)

Заключение

В ходе выполнения расчётно-графической работы спроектирована и реализована реляционная база данных «Мониторинг сети» в СУБД PostgreSQL.

Выполненные задачи:

- Спроектирована логическая структура БД из 6 взаимосвязанных таблиц: nodes, metric_types, metrics, incidents, engineers, alerts.
- Разработан скрипт инициализации schema.sql с полным набором ограничений целостности: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK.
- Внедрены расширенные ограничения CHECK для реализации бизнес-логики (допустимые статусы, неотрицательные значения метрик, контроль времени закрытия аварий).
- Создано 3 индекса для оптимизации частых операций поиска и фильтрации.
- База данных наполнена тестовыми данными (49 записей суммарно), достаточными для демонстрации работоспособности запросов.
- Разработан комплекс из 10 функциональных SQL-запросов, демонстрирующих возможности системы: простые выборки, фильтрация, агрегация с GROUP BY, HAVING, JOIN-отчёты, CTE, сортировка с ограничением выборки, аналитика по временным периодам.
- Построена ER-диаграмма логической структуры БД.
- Подготовлен файл README.md с описанием проекта и обоснованием выбора ограничений и стратегии индексации

